

NeuroGuardian X Software Stack Report

Explanation of all major languages, tools, frameworks, libraries, frontend modules, backend services, and why each one is used.

Prepared date	September 3, 2026
Project	NeuroGuardian X wearable safety companion
Mobile version	0.3.0+3
Main software goal	Connect ESP32-S3 wearable data to an Android app, detect safety risk, and send SOS alerts with location.

SIMPLE SUMMARY

Flutter/Dart builds the phone app, Python/FastAPI runs the emergency backend, Arduino C++ runs the ESP32-S3 wearable firmware, and JSON connects all layers together.

1. Main Languages Used

Language	Where used	Why we used it
Dart	Flutter Android app	Flutter apps are written in Dart. It gives fast UI updates, clean app logic, and good async support for BLE/GPS/network calls.
Python	Emergency backend	FastAPI with Python is simple, readable, quick to build, and strong for API integrations like Google Places and WhatsApp.
Arduino C++	ESP32-S3 firmware	ESP32 sensor, BLE, buzzer, vibration, battery, and GPS logic needs low-level hardware control.
JSON	Watch-app-backend data exchange	Easy for ESP32, Flutter, and Python to read. It keeps vitals packets and SOS payloads simple and clear.
YAML	Flutter project config	Used by pubspec.yaml to define app dependencies, version, assets, and SDK rules.
PowerShell	Windows build/install scripts	Used to run Flutter builds, APK installs, backend launch scripts, and desktop helper commands.

2. Frontend: Flutter Mobile App

The frontend is the Android app used by the patient/wearable owner. It shows live health data, risk detection, SOS countdown, device connection, care suggestions, history, and settings.

- Location: C:\Users\hp\Downloads\neuro guardian x\neuroguardian_app\lib
- Main role: user interface, Bluetooth listener, SOS timer, GPS fetch, local data storage, and emergency request sender.
- Reason for Flutter: one clean codebase can create a real Android app with a polished UI and hardware-connected features.

Frontend package/tool	Purpose in app	Why it was chosen
flutter_blue_plus	Scans and connects to ESP32-S3 over BLE	Needed for receiving wearable vitals and sending commands like BUZZER_ON/BUZZER_OFF.
flutter_riverpod	App state management	Keeps live vitals, settings, SOS state, and UI updates organized.
geolocator	Phone GPS location	Gets emergency location when watch GPS is missing or weak.
http	Backend API calls	Sends SOS payloads and gets nearby hospital data from backend.
url_launcher	Open Maps/WhatsApp/SMS links	Gives safe fallback options when automatic provider sending is unavailable.
hive and hive_flutter	Local phone database	Stores settings, prescriptions, alert history, and local records without a server database.
fl_chart	Charts and analytics	Used for health graphs and trend visualizations.
flutter_local_notifications	Local phone alerts	Supports phone-side notifications for emergency/user attention.
Material UI	App visual components	Provides reliable Android UI elements like tabs, cards, buttons, forms, and navigation.

3. Frontend App Modules

Module	File area	What it does
Dashboard	features/dashboard	Shows live vitals, risk detection, temperature meter, emergency status, and wearable state.
Device	features/device and features/ble	Handles scan/connect/reconnect flow for the ESP32-S3 wearable.
SOS	features/emergency	Runs manual SOS, auto SOS, GPS lookup, nearby hospital lookup, and backend notification call.
Care	features/care	Shows minor care suggestions, serious consult-doctor warning, and doctor-entered prescriptions.
History	features/history and features/alerts	Stores and displays past alerts, fall events, and emergency actions.
Settings	features/settings	Stores guardian WhatsApp number, ambulance contact, backend URL, backend token, and push target.
Metrics	features/metrics	Represents vitals data received from hardware and shares it across the app.
Knowledge	features/knowledge	Contains condition database cards for heart, stress, temperature, fall, and related health criteria.

4. Backend: Emergency API Server

The backend protects secret keys and handles outside services. The mobile app calls the backend during an SOS, and the backend finds nearby hospitals and sends notifications.

- Main file: C:\Users\hp\Downloads\neuro guardian x\neuroguardian_app\backend\emergency_server.py
- Configuration: C:\Users\hp\Downloads\neuro guardian x\neuroguardian_app\backend\.env stores secrets; keys are not placed inside app code.
- Main endpoints: /health, /medical-facilities/nearby, and /emergency/alert.

Backend tool/library	Purpose	Why it was chosen
FastAPI	Creates emergency REST API endpoints	Fast, clean, modern Python API framework with automatic validation support.
Uvicorn	Runs the backend server	Standard lightweight server for FastAPI apps.
Pydantic	Validates SOS, location, vitals, guardian, and hospital payloads	Prevents malformed data from entering notification logic.
httpx	Calls external APIs	Used for Google Places, Meta WhatsApp Cloud API, Twilio, SuperSend, and ntfy requests.
python-dotenv	Loads .env values	Keeps API keys and tokens outside source code.

5. External Services Used

Service	Used for	Why we used it
Google Places API	Finds top 3 nearby hospitals/medical facilities	Provides real nearby medical facility data based on patient GPS.

Service	Used for	Why we used it
Google Maps link	Shares exact patient location	Guardian can tap the link and open navigation immediately.
Meta WhatsApp Cloud API	Automatic WhatsApp message to guardian	Official WhatsApp sending path, but template approval is required.
Twilio WhatsApp	Optional WhatsApp backup provider	Useful if Meta direct setup is difficult or production needs managed delivery tools.
ntfy push	Push notification fallback	Simple backup alert channel when WhatsApp is not ready.
SuperSend	Optional notification provider	Kept as future/optional notification route if account endpoints are available.

6. Firmware Software Part

The firmware is software running inside the ESP32-S3 watch. It reads sensors, detects hardware-side events, sends BLE packets, and responds to commands from the phone app.

Firmware tool/library	Purpose	Why it was used
Arduino ESP32 core	Runs ESP32-S3 code	Simple embedded development environment with strong ESP32 support.
ESP32 BLE libraries	BLE service, notify characteristic, command characteristic	Required for watch-app Bluetooth communication.
Wire/I2C	Sensor communication bus	Needed for IMU, optical heart sensor, and temperature modules.
MPU6050 logic	Motion and fall detection	Low-cost prototype IMU for acceleration and gyro readings.
MAX30105/MAX30102 optional library	Heart rate and SpO2	Adds optical vitals when the sensor is wired.
TinyGPSPlus optional library	GPS parsing	Adds watch-side GPS if GPS module is installed.
GPIO control	Buzzer, vibration motor, LED, button, battery input	Direct hardware control for emergency feedback and manual SOS.

7. Data Flow Between Frontend, Backend, and Wearable

SYSTEM FLOW ESP32-S3 watch -> BLE JSON packet -> Flutter app -> risk/SOS logic -> GPS -> backend -> Google Places -> WhatsApp/push/SMS fallback.			
Step	Input	Processing	Output
1. Wearable	Sensor readings and fall/no-movement state	ESP32-S3 packages readings as JSON over BLE	Live vitals packet
2. App	BLE packet and user settings	Updates dashboard and checks emergency rules	Countdown, I AM OKAY, or SOS
3. Location	Watch GPS or phone GPS	Chooses best available location	Google Maps exact-location link
4. Backend	SOS payload, location, guardian target	Finds nearby hospitals and formats message	Alert response and facility list
5. Notifications	Guardian number/push target	Sends WhatsApp/push where configured	Guardian receives emergency message

8. Database and Storage

Storage/database	Where	Used for
Hive local database	Phone app	Stores settings, alert history, prescriptions, and local user data.
Static condition database	Flutter code	Stores health-condition cards, minor guidance, and serious warning logic.
Google Places database	External Google API	Returns nearby hospital/medical facility information.
PhysioNet datasets	Future AI/model training	MIT-BIH, sudden cardiac death, wearable, and stress datasets for research and model building.
Prototype watch logs	Future collection	Most important real-world data for threshold tuning and AI validation.

9. Why This Stack Is Suitable

- Flutter gives a polished Android app quickly and supports BLE, GPS, local storage, charts, and notifications.
- Python FastAPI is a good backend choice because emergency APIs and external-service calls are easy to build and maintain.
- ESP32-S3 is low-cost, BLE-ready, and suitable for a prototype wrist-watch safety device.
- JSON keeps communication simple across embedded firmware, mobile app, and backend.
- Keeping secrets in the backend protects Google/WhatsApp keys from being exposed inside the APK.
- The stack can grow into production by adding live tracking, verified emergency dispatch, cloud hosting, logging, and tested AI models.

10. Current Software Limitations

- SOS currently sends the current location at trigger time; continuous live location tracking still needs to be added.
- WhatsApp automatic alerts depend on approved Meta/Twilio templates and configured guardian numbers.
- Hospital/ambulance direct dispatch is not the same as Google Maps hospital lookup; verified service integration is needed.
- Real AI model training is still future work; current detection is mainly rule-based safety logic.
- Real sensors must be wired, calibrated, tested, and validated before public release.